

LABORATÓRIO: PRIMEIRO PROGRAMA EM JAVA COM O ASSISTENTE APACHE MAVEN PADRÃO NO IDE APACHE NETBEANS E PORTABILIDADE DE PROJETOS MAVEN ENTRE IDEs

Última atualização: 13/04/2024

PARTE I – APACHE NETBEANS COM APACHE MAVEN

1. Crie um projeto Java no **IDE APACHE NETBEANS** que seja baseado na ferramenta de *build* **Apache Maven** (Figuras 1 e 2).

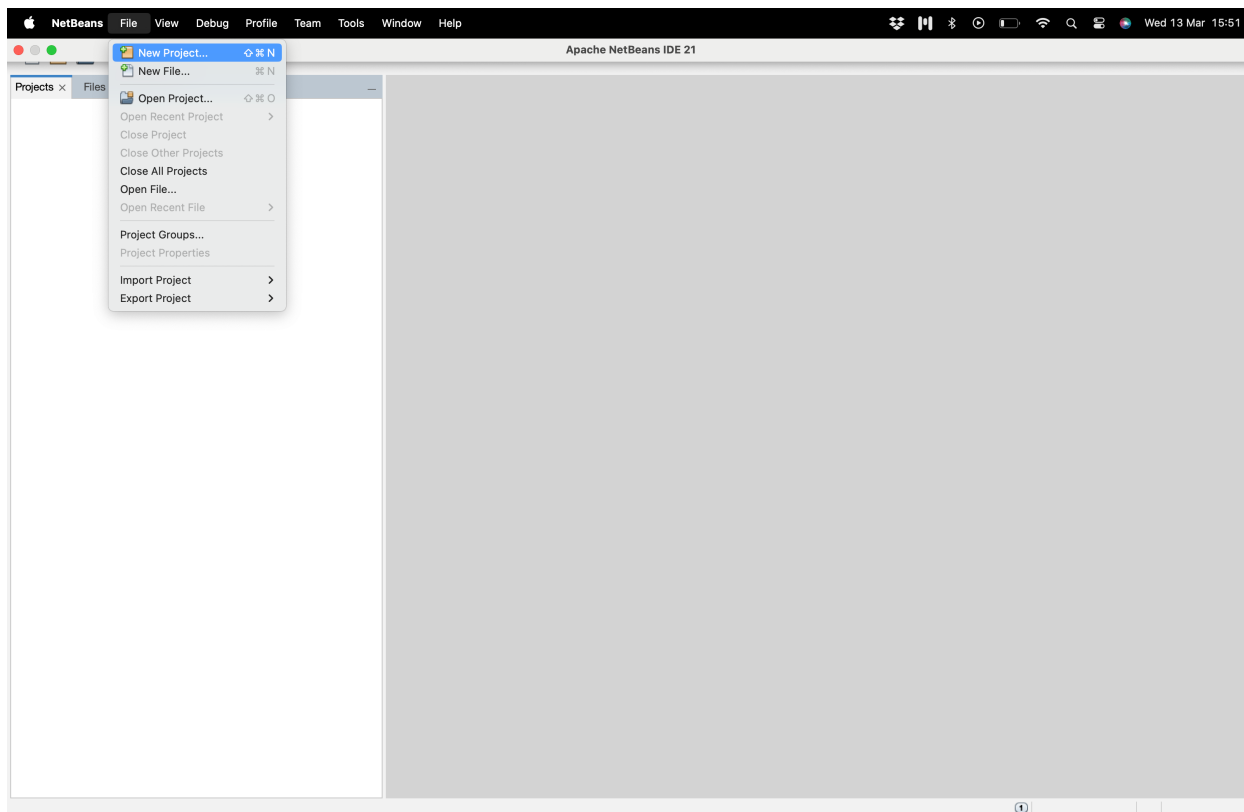


Figura 1. Criação de um novo projeto no IDE Apache NetBeans.

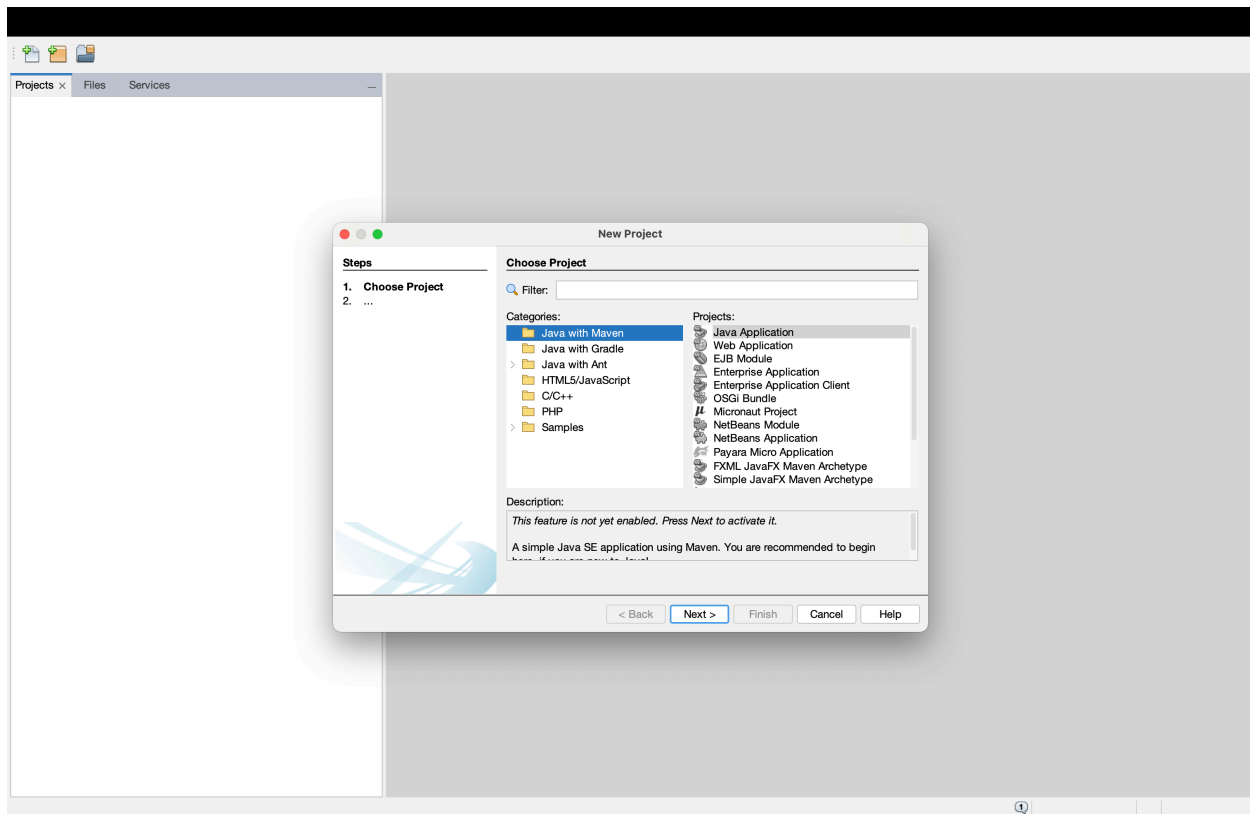


Figura 2. Selecione Java With Maven – Java Application para que seja criado um projeto em Java baseado no Apache Maven.

3. Conforme pode ser visto na Figura 3, escolha um nome para o projeto (*Project Name*) e uma localização no sistema de arquivos onde o projeto será criado (*Project Location*). Não é necessário alterar os demais campos deste assistente (*Artifact Id*, *Group Id*, *Version* e *Package*).

New Java Application

Steps

1. Choose Project
- 2. Name and Location**

Name and Location

Project Name:

Project Location:

Project Folder:

Artifact Id:

Group Id:

Version:

Package: (Optional)

< Back Next > **Finish** Cancel Help

Figura 3. Configuração do Nome e local do projeto.

4. Conforme pode ser visto na Figura 4, observe que já é gerada automaticamente uma classe com um método `main` (classe `LaboratorioNetBeansMaven.java`)

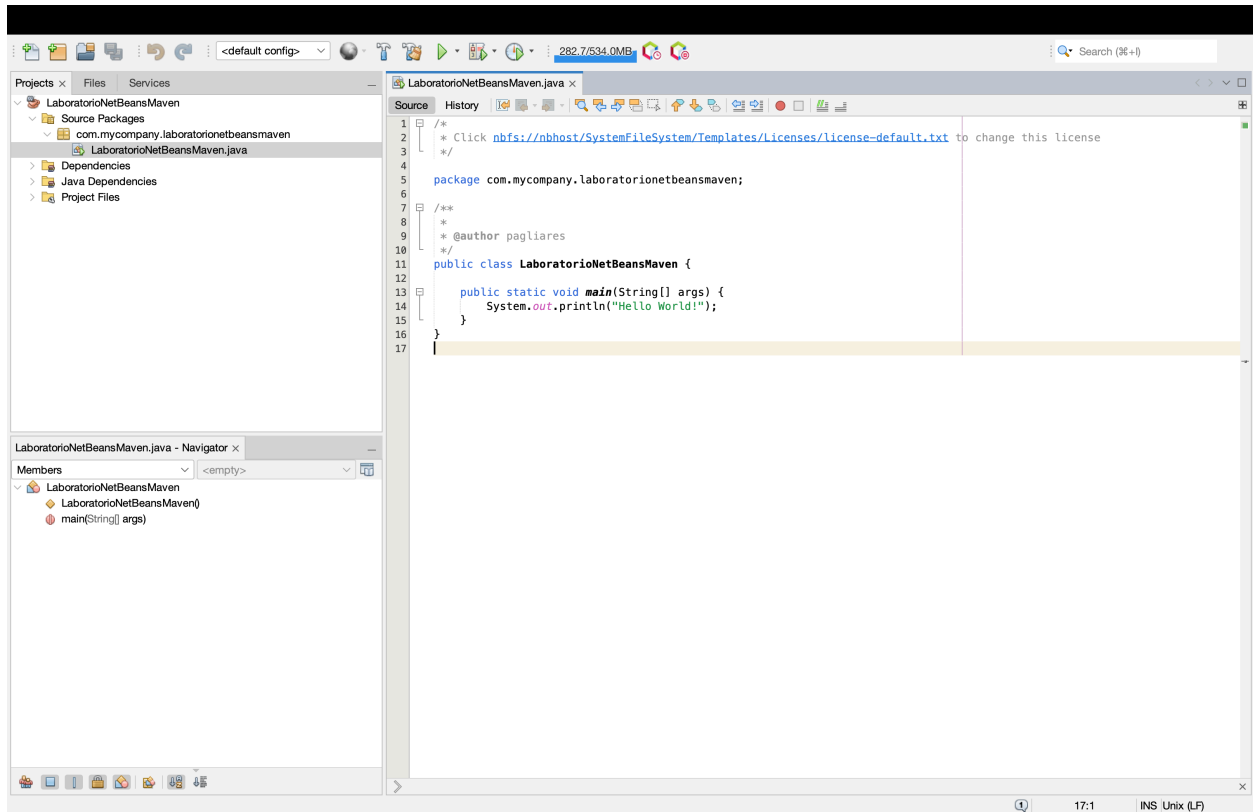


Figura 4. O projeto Java com Maven é criado.

5. Altere o conteúdo do método main de forma a exibir a mensagem "Hello, cruel Apache NetBeans and Maven world!".

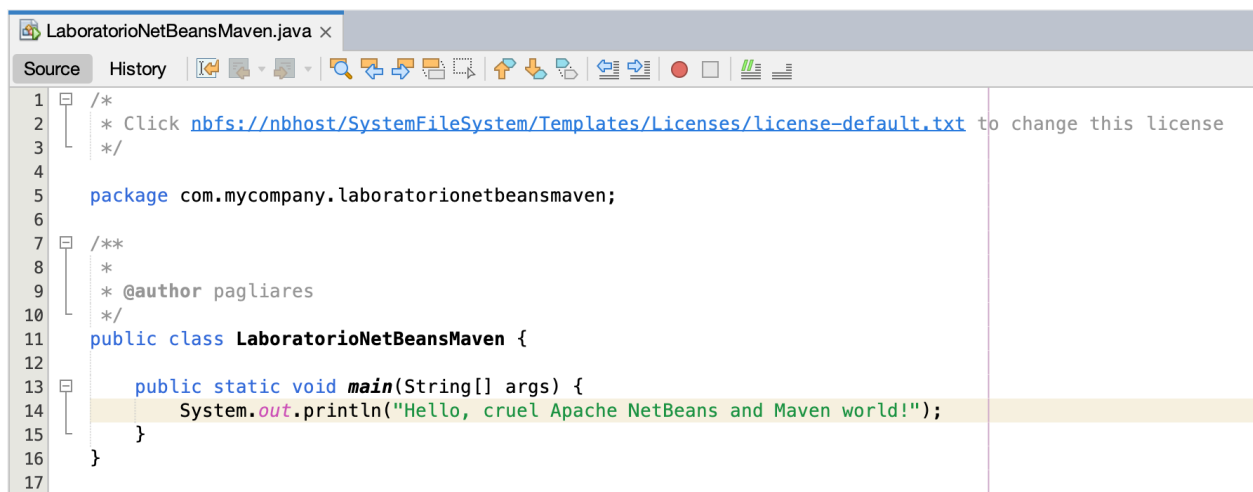


Figura 5. Método main com a nova mensagem a ser exibida no console.

6. Salve o projeto. Agora você já está pronto para executar o projeto (Seta verde na parte superior da Figura 6).

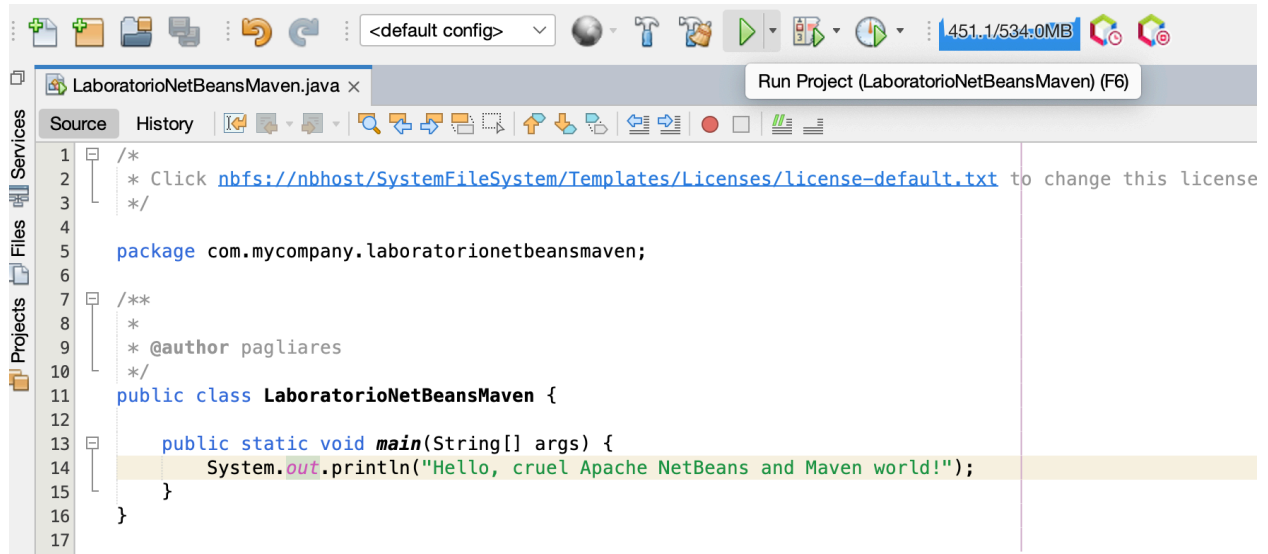


Figura 6. Executando o projeto no Apache NetBeans.

7. Repare que na saída (Figura 7) está descrito que foi feito o *build* da aplicação a partir do Apache Maven (que lê durante o processo o arquivo `pom.xml`)

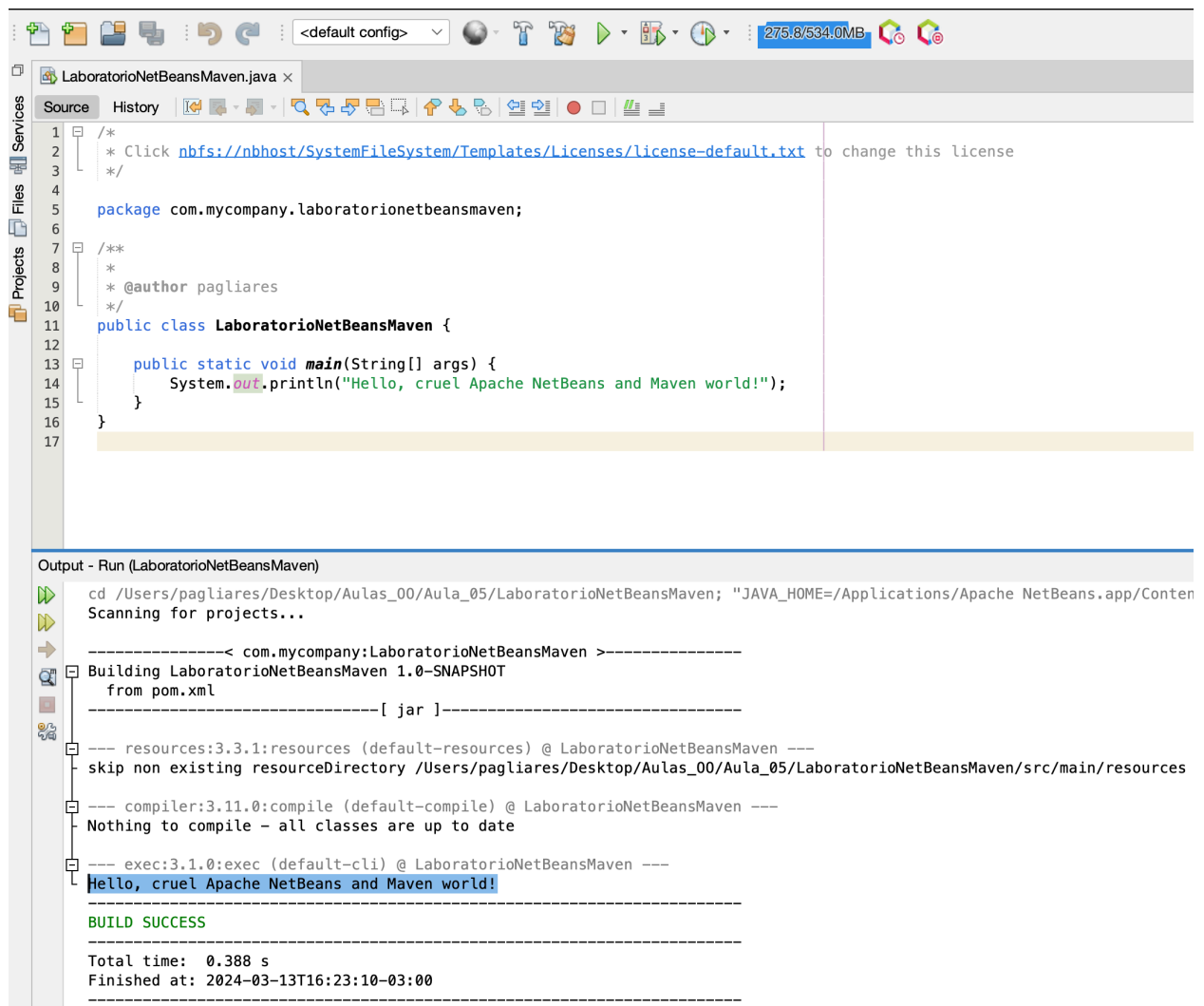


Figura 7. Saída exibida após compilação e execução do projeto (*build* feito com o Apache Maven).

8. Abra o arquivo `pom.xml`. Ele é o principal arquivo de configuração do Apache Maven. O arquivo `pom.xml` é o principal arquivo de configuração do Apache Maven. Observe que é nele que configuramos o JDK a ser utilizado (JDK 21 no meu exemplo)

```
<maven.compiler.source>21</maven.compiler.source>
```

```
<maven.compiler.target>21</maven.compiler.target>
```

Também é no arquivo `pom.xml` que a tríade de configuração do Apache-Maven reside (repare que a configuração desta tríade foi feita na Figura 3, quando criamos o projeto):

```
<groupId>com.mycompany</groupId>
```

```
<artifactId>LaboratorioNetBeansMaven</artifactId>
```

```
<version>1.0-SNAPSHOT</version>
```

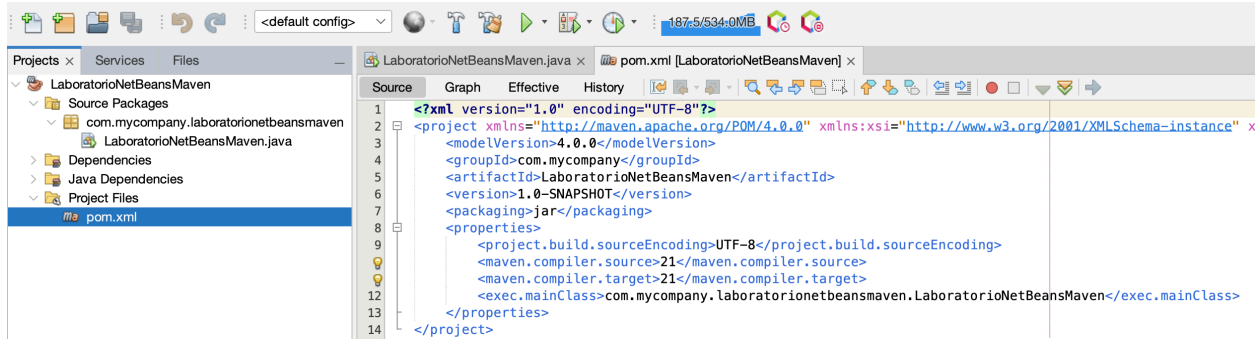


Figura 8. O principal arquivo de configuração do Apache Maven (`pom.xml`).

Existem outras ferramentas de build para Java além do Apache Maven. A ferramenta Gradle é a mais conhecida entre as alternativas. O uso de **Maven** ou **Gradle** é a opção recomendada por fornecer diversas vantagens, dentre elas gestão de dependências, *build* customizável, estrutura padrão de projeto portátil entre IDEs, etc. Usaremos **Maven** neste curso.

PARTE II – INTELIJ IDEA COMMUNITY/ULTIMATE EDITION (JETBRAINS)

1. Importe o projeto compatível com **Maven** criado na Parte I deste laboratório no IDE IntelliJ.

Dica: Selecione a opção “Abrir” (*Open*) na indique o caminho para o diretório do projeto que contém o arquivo `pom.xml` (Veja Figura 9)

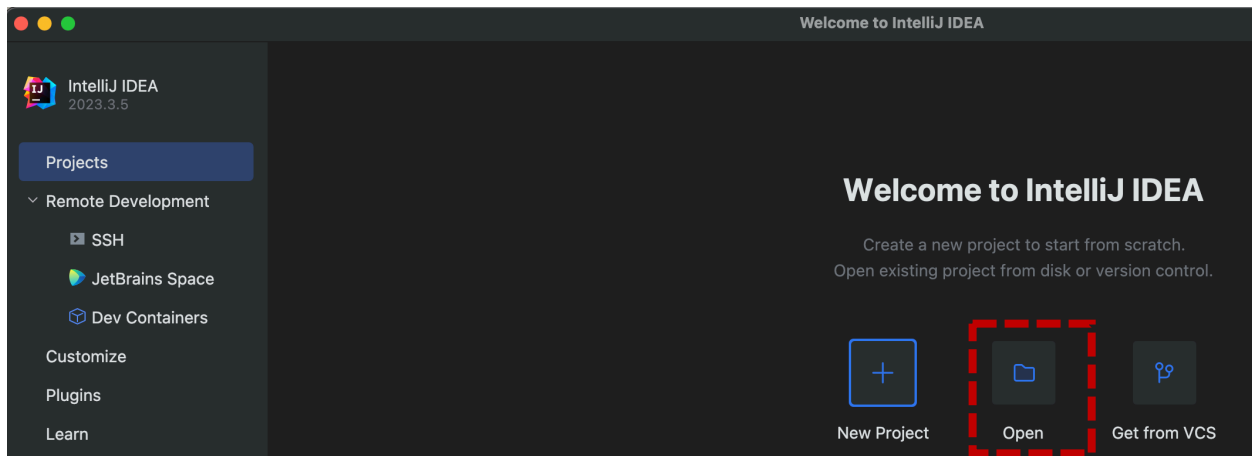
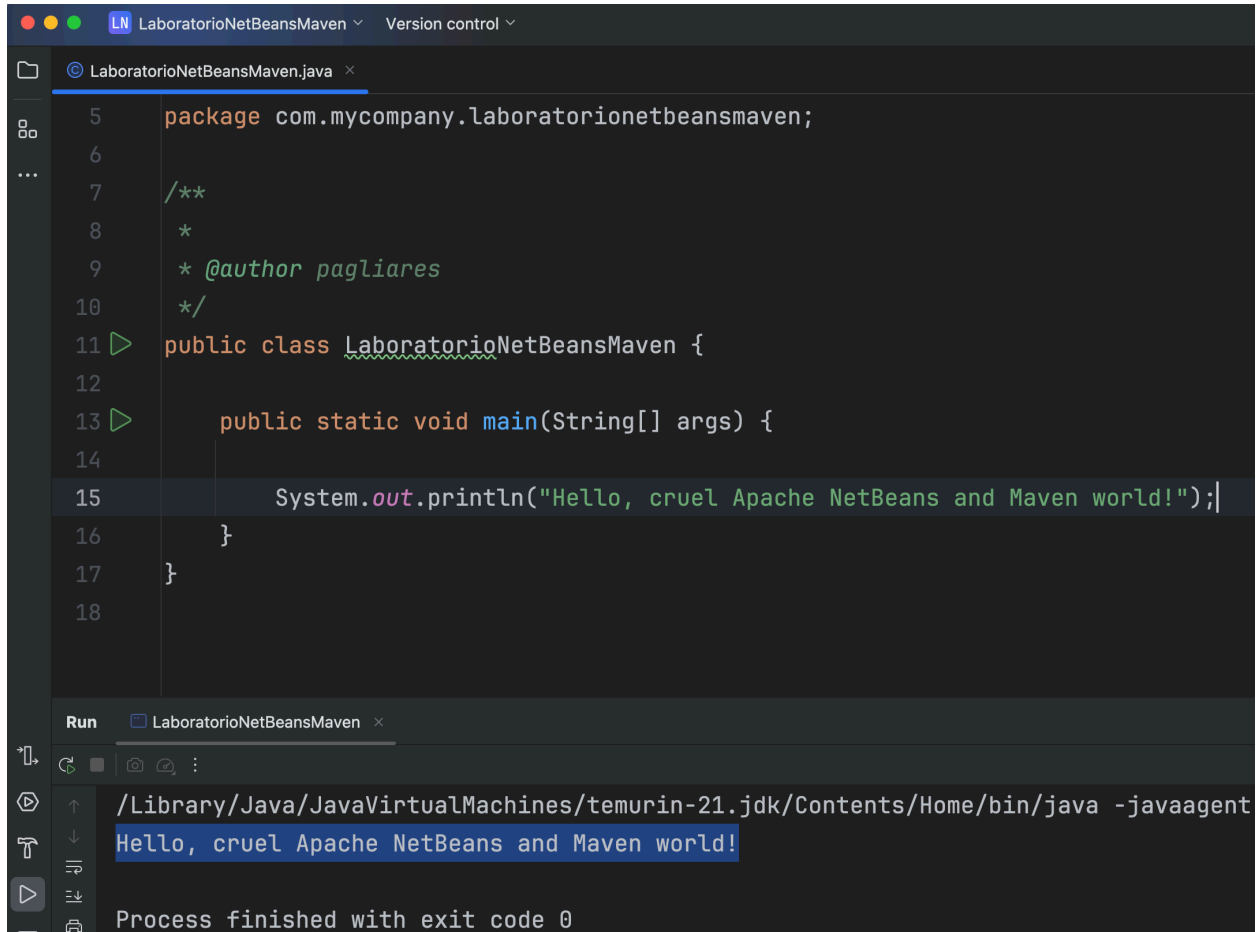


Figura 9. Tela inicial do IDE IntelliJ IDEA Ultimate Edition.

2. Execute o projeto compatível com **Maven** criado na Parte I deste laboratório no **IDE IntelliJ**. **Dica:** Uma das formas de executar o projeto é clicar na seta verde – Linha 11 – na classe que contém o método `main` da aplicação (Veja Figura 10). Observe que existe portabilidade entre IDEs de projetos criados com Apache Maven.



```
5 package com.mycompany.laboratorionetbeansmaven;
6
7 /**
8  *
9  * @author pagliares
10 */
11 public class LaboratorioNetBeansMaven {
12
13     public static void main(String[] args) {
14
15         System.out.println("Hello, cruel Apache NetBeans and Maven world!");
16     }
17 }
18
```

Run LaboratorioNetBeansMaven

/Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home/bin/java -javaagent
Hello, cruel Apache NetBeans and Maven world!

Process finished with exit code 0

Figura 10. Executando o projeto Maven no IntelliJ IDEA.

3. Feche o projeto. Você irá agora criar um projeto Java com suporte a Maven (dessa vez no IntelliJ). O seu projeto deverá exibir no console a mensagem "Hello, cruel IntelliJ and Maven world!". Lembre-se de selecionar a opção do assistente do **IntelliJ** para criação de um projeto Java com Maven em vez de apenas importar um projeto Maven como feito, por exemplo, no exercício anterior (Veja Figura 11).

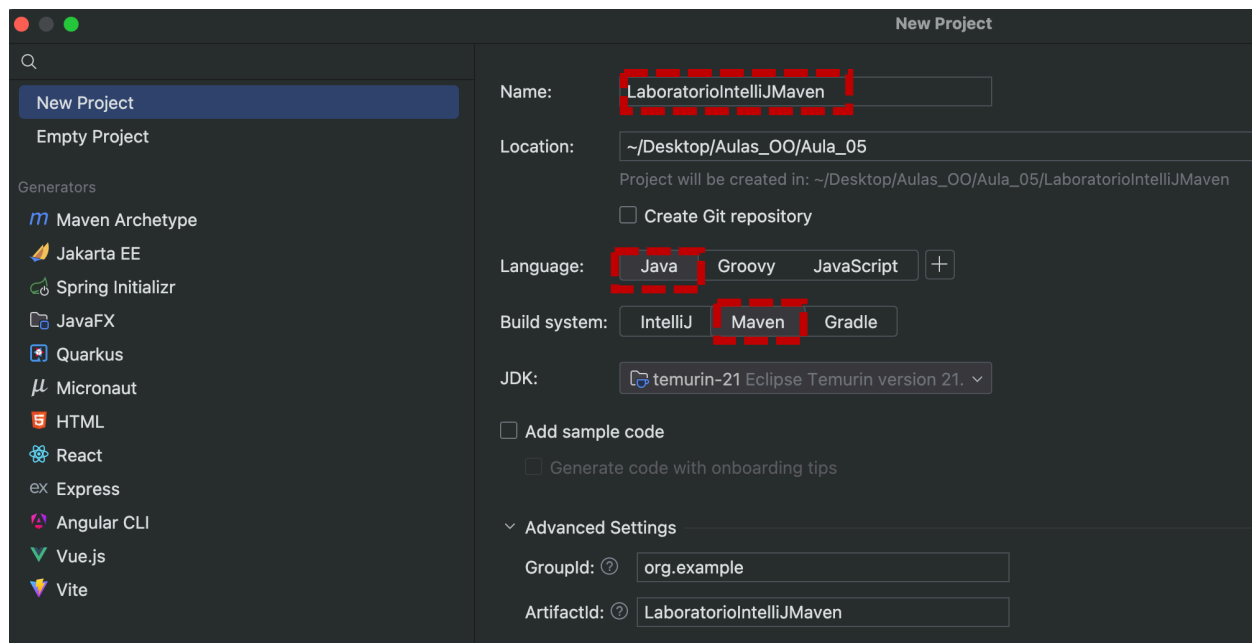


Figura 11. Criação de um projeto Java com suporte a Apache Maven no IDE IntelliJ IDEA Ultimate Edition.

4. Execute o projeto. Sua saída deverá ser similar à saída exibida na Figura 10, exibindo Hello, cruel IntelliJ and Maven world! Em vez de NetBeans.

PARTE III – IDE ECLIPSE FOR JAVA DEVELOPERS (ECLIPSE FOUNDATION)

1. Desta vez eu não vou fornecer as telas como nos exercícios anteriores. **Importe e execute** o projeto compatível com **Maven** criado na Parte II deste laboratório no IDE Eclipse. **Dica:** Observe que existe uma opção no **Eclipse** para importar projetos **Maven**. Mais uma vez você deverá perceber que existe portabilidade de projetos Java com Maven entre IDEs.

2. Feche o projeto. Crie um projeto Java com suporte a Apache Maven. Uma vez criado o projeto, implemente no **Eclipse** um programa que exiba a mensagem "Hello, cruel Eclipse and Maven world!". Deve ser usado o assistente do **Eclipse** para criação de um projeto Java com **Maven** em vez de apenas importar um projeto Maven como feito, por exemplo, no exercício anterior..

PARTE IV – O ARQUIVO POM.XML DO APACHE MAVEN

1. O arquivo `pom.xml` é o principal artefato usado pelo **Maven** para o *build* das aplicações **Java**. Certifique-se, verificando os arquivos `pom.xml`, que em todos os projetos criados neste Laboratório (**Eclipse**, **IntelliJ** e **NetBeans**) estejam configurados para usar a versão da linguagem **Java** de acordo com o JDK instalado em sua máquina (ou no laboratório). No meu caso, eu usei JDK 21 e meu arquivo `pom.xml` possui a seguinte configuração:

```
<properties>  
  <maven.compiler.source>21</maven.compiler.source>  
  <maven.compiler.target>21</maven.compiler.target>  
</properties>
```

Dica: para saber a versão default instalada em sua máquina, acesso o terminal/prompt e digite
java -version.